



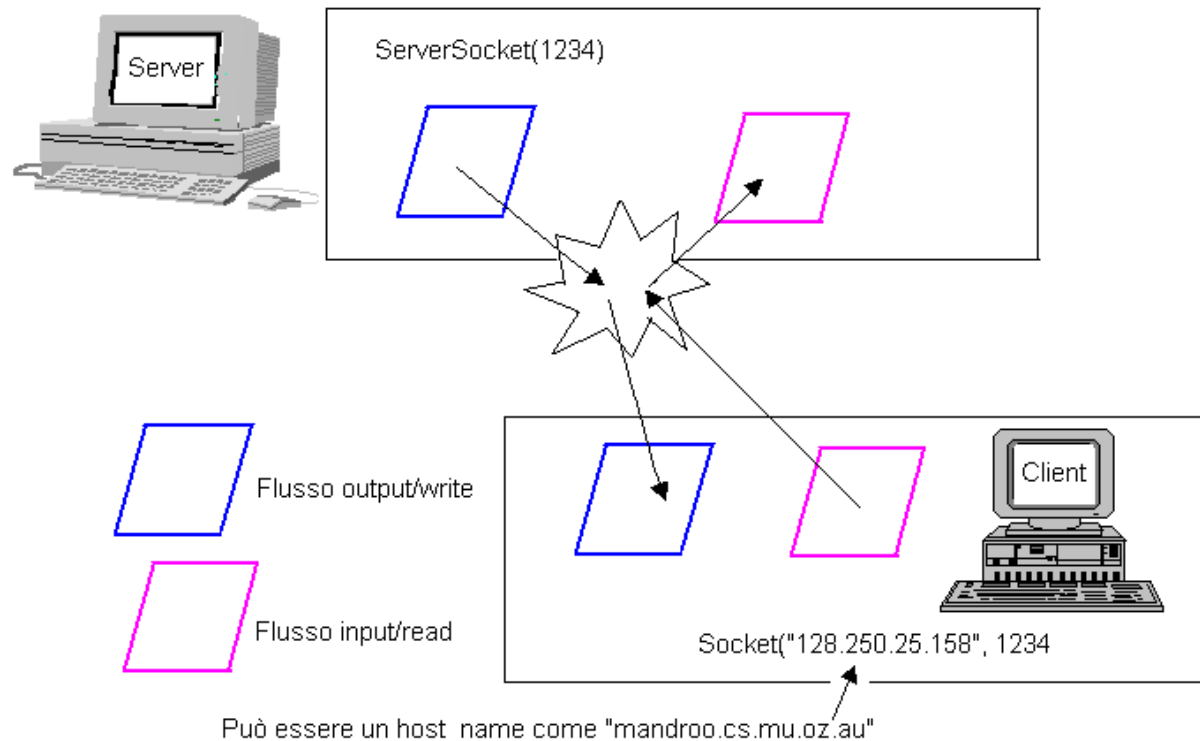
Socket Java



Reti di Calcolatori Modulo 2

Socket Java (TCP)

- ▶ Il pacchetto .Net di Java fornisce due classi
 - ▶ Socket: per implementare un client
 - ▶ ServerSocket: per implementare un server



Socket Java (TCP)

- ▶ Due passi
 - ▶ Creazione di un server che si mette in ascolto
 - ▶ Creazione di un client che stabilisce la connessione
- ▶ La comunicazione avviene direttamente attraverso i canali messi a disposizione dall'oggetto Socket
 - ▶ `getInputStream()`, `getOutputStream()`, ottengono rispettivamente un canale di input e di output

Socket Java: Server

1. Creare un oggetto `ServerSocket`, specificando il numero di porta a cui legarlo
 - ▶ `ServerSocket server;`
 - ▶ `DataOutputStream os;`
 - ▶ `DataInputStream is;`
 - ▶ `Server = new ServerSocket( PORT );`
2. Aspettare la richiesta di connessione dal client
 - ▶ `Socket clientsock = server.accept();`
 - ▶ Quando `accept()` termina la connessione col client è stabilita e restituisce un socket per comunicare
3. Creare flussi I/O per comunicare con il client
 - ▶ `is = new DataInputStream(clientsock.getInputStream());`
 - ▶ `os = new DataOutputStream(clientsock.getOutputStream());`

Socket Java: Server

4. Eseguire la comunicazione con il client
 - ▶ Ricevere i dati dal client: `String line = is.readLine();`
 - ▶ Inviare i dati al client: `os.writeBytes("Hello\n");`
5. Alla fine chiudere il socket
 - ▶ `client.close();`

Socket Java: Client

1. Creare un oggetto di tipo Socket, specificando indirizzo e porta
 - ▶ `client = new Socket ( server, port_id );`
 - ▶ Quando l'oggetto è costruito la connessione col server è stabilita
2. Creare flussi I/O per comunicare con il server
 - ▶ `is = new DataInputStream( client.getInputStream() );`
 - ▶ `os = new DataOutputStream( client.getOutputStream() );`
3. Eseguire I/O o la comunicazione con il server
 - ▶ ricevere dati dal server: `String line = is.readLine();`
 - ▶ inviare i dati al server: `os.writeBytes("Hello\n");`
4. Alla fine chiudere il socket
 - ▶ `client.close();`



Socket Java: Serializzazione

- ▶ Il meccanismo della serializzazione permette di scrivere e ricevere direttamente oggetti (istanze di classi) sui flussi in output e input
- ▶ Gli oggetti scambiati devono implementare l'interfaccia Serializable
- ▶ La maggior parte delle classi standard implementa Serializable

Socket Java: Server

```
//SimpleServer.java: un semplice programma
server
import java.net.*;
import java.io.*;

public class SimpleServer {
    public static void main(String args[])
        throws IOException {

//Registra il servizio sulla porta 1234
    ServerSocket s = new ServerSocket(1234);

//Aspetta e accetta una connessione
    Socket s1=s.accept();
```



Socket Java: Server

```
// Ottiene un flusso di comunicazione associato al
// socket
OutputStream slout = s1.getOutputStream();
DataOutputStream dos = new DataOutputStream(slout);

// Invia una stringa!
dos.writeUTF("Hello world");

// Chiude la connessione, ma non il socket server
dos.close();
slout.close();
s1.close();
}
}
```

Socket Java: Client

```
// SimpleClient.java: un semplice programma client
import java.net.*;
import java.io.*;

public class SimpleClient {
    public static void main(String args[])
        throws IOException {

// Apre la connessione ad un server, porta 1234
Socket s1 = new Socket("mysrv.dti.unimi.it", 1234);
```



Socket Java: Client

```
// Ottiene un file handle dal socket e legge l'input
InputStream s1In = s1.getInputStream();
DataInputStream dis = new DataInputStream(s1In);
String st = new String (dis.readUTF());
System.out.println(st);

// Alla fine, chiude la connessione e esce
dis.close();
s1In.close();
s1.close();
}
}
```



Socket: Eccezioni

```
try {  
    Socket client = new Socket(host, port);  
    handleConnection(client);  
}  
catch (UnknownHostException uhe) {  
    System.out.println("Unknown host: " +  
host);  
    uhe.printStackTrace();  
}  
catch (IOException ioe) {  
    System.out.println("IOException: " + ioe);  
    ioe.printStackTrace();  
}
```

Socket: Eccezioni

- ▶ `public ServerSocket(int port) throws IOException`
 - ▶ Crea un socket server su una porta specificata
 - ▶ Un numero di porta 0 crea un socket sulla prima porta libera
 - ▶ Si può poi usare `getLocalPort()` per identificare la porta su cui il socket è in ascolto
 - ▶ La lunghezza massima della coda per i three-way handshake in sospeso (richieste di connessione) è impostata a 50
 - ▶ Se una richiesta di connessione arriva quando la coda è piena, la connessione viene rifiutata

Socket: Eccezioni

- ▶ throws

- ▶ IOException

- Se si verifica un errore I/O quando si apre il socket

- ▶ SecurityException

- Se esiste un manager della sicurezza e il suo metodo `checkListen` non consente il funzionamento

Server in Loop

```
// SimpleServerLoop.java: un semplice programma server che
// gira per sempre in un singolo thread
import java.net.*;
import java.io.*;
public class SimpleServerLoop {
    public static void main(String args[]) throws IOException
    {
        // Register service on port 1234
        ServerSocket s = new ServerSocket(1234);
        while(true)
        {
            Socket s1=s.accept(); // Aspetta e accetta una
                                   // connessione
        }
    }
}
```



Server in Loop

```
// Ottiene un flusso di comunicazione associato al
//socket
OutputStream slout = s1.getOutputStream();
DataOutputStream dos = new DataOutputStream (slout);

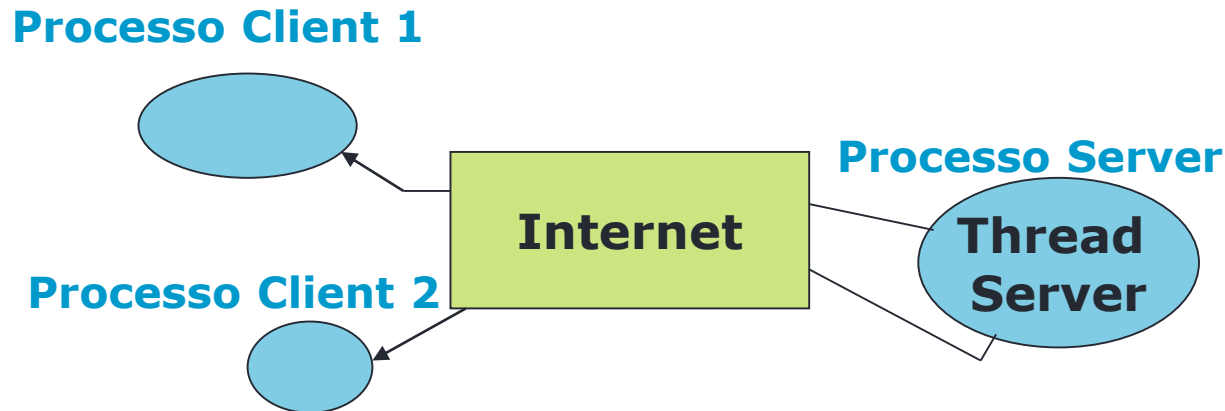
// Manda una stringa!
dos.writeUTF("Hi there");

// Chiude la connessione, ma non il socket server
dos.close();
slout.close();
s1.close();
}
}
```



Socket Multithread

- ▶ Per server multithread:
 - ▶ while (true)
 - ▶ aspettare le richieste del client (v. fase 2)
 - ▶ creare un thread con socket “sock” come parametro
 - ▶ Il thread crea flussi (v. fase 3) ed esegue la comunicazione (v. fase 4)
 - ▶ fornito il servizio eliminare il thread



Esempio socket paralleli: thread

```
class ServerSocket{
    ...
    public void listenSocket(){
    ...
        while(true){
            ClientWorker w;
            try{
                sock = server.accept();
                w = new ClientWorker(sock);
                Thread t = new Thread(w);
                t.start();
            }
        }
    }
}
```

Server

```
class ClientWorker implements Runnable {
    private Socket client;

    ClientWorker(Socket client) {
        this.client = client;
    }

    public void run(){
        String line;
        BufferedReader in = null;
        PrintWriter out = null;
        try{
            in = new BufferedReader(new
                InputStreamReader(client.getInputStream()));
            out = new
                PrintWriter(client.getOutputStream(), true);
            ...
        } catch (IOException e) {
            System.out.println("in or out failed");
            System.exit(-1);
        }
    }
}
```

Client



Annotazioni finali

► References:

- MSDN – Getting started with Winsock
(http://msdn.microsoft.com/library/default.asp?url=/library/en-us/winsock/winsock/getting_started_with_winsock.asp)

Socket Java (UDP)

- ▶ Nessuna connessione
- ▶ Specifica indirizzo e porta destinatario
- ▶ Invio diretto

Socket Java: Client

```
import java.io.*;
import java.net.*;

class UDPClient
{
    public static void main(String args[]) throws Exception
    {
        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));
        DatagramSocket clientSocket = new DatagramSocket();
        InetAddress IPAddress = InetAddress.getByName("localhost");
        byte[] sendData = new byte[1024];
        byte[] receiveData = new byte[1024];
        String sentence = inFromUser.readLine();
        sendData = sentence.getBytes();
    }
}
```



Socket Java: Client

```
DatagramPacket sendPacket = new DatagramPacket(sendData,  
    sendData.length, IPAddress, 9876);  
clientSocket.send(sendPacket);  
DatagramPacket receivePacket = new  
    DatagramPacket(receiveData, receiveData.length);  
clientSocket.receive(receivePacket);  
String modifiedSentence = new String(receivePacket.getData());  
System.out.println("FROM SERVER:" + modifiedSentence);  
clientSocket.close();  
}  
}
```



Socket Java: Server

```
▶ import java.io.*;
import java.net.*;

class UDPServer
{
    public static void main(String args[]) throws Exception
    {
        DatagramSocket serverSocket = new DatagramSocket(9876);
        byte[] receiveData = new byte[1024];
        byte[] sendData = new byte[1024];
        while(true)
        {
            DatagramPacket receivePacket = new
                DatagramPacket(receiveData, receiveData.length);
            serverSocket.receive(receivePacket);
            String sentence = new String( receivePacket.getData());
            System.out.println("RECEIVED: " + sentence);
            InetAddress IPAddress = receivePacket.getAddress();
            int port = receivePacket.getPort();
```

Socket Java: Server

```
String capitalizedSentence = sentence.toUpperCase();
sendData = capitalizedSentence.getBytes();
DatagramPacket sendPacket =
    new DatagramPacket(sendData, sendData.length,
        IPAddress, port);
serverSocket.send(sendPacket);
}
}
}
```